Project Report - ECE 176

Alexey Grishakov
Electrical and Computer engineering
A17999821

Abstract

This project tests whether a Grassmann-flow language model can serve as an attention-free alternative to a standard causal Transformer at approximately 100M parameters. Instead of token-to-token self-attention, the Grassmann model uses causal geometric mixing with Plücker coordinates computed from low-dimensional token subspaces. Both models use the GPT-2 tokenizer and the same training setup, so the comparison mainly isolates the effect of the mixing mechanism itself.

At step $9{,}500$ (about 10 billion tokens, using the same dataloaders for both models), the Grassmann model achieves loss 5.8413 and perplexity 344.23, compared with Transformer loss 6.1234 and perplexity 456.41. Total training time was $12{:}42$ for Grassmann and $24{:}06$ for the Transformer, making Grassmann roughly $1.9\times$ faster to train. However, profiling inference with KV caching shows only a modest compute difference between the two, so the main advantage in these experiments is better quality and faster training rather than a clear inference-cost reduction.

1 Introduction

Most autoregressive language models today rely on self-attention, which scales quadratically with sequence length. As context windows and model sizes increase, this becomes expensive in both memory and compute. The central question of this project is whether attention can be replaced entirely by a geometric flow mechanism while still preserving useful language-modeling quality.

The motivation for this project is to test whether an attention-free architecture can retain strong language-modeling performance while reducing the training and long-context costs associated with standard self-attention. If a geometric mixer can match or exceed a Transformer at similar scale, it could offer a more efficient alternative for future long-context language models.

The architecture studied here comes from "Attention Is Not What You Need" [1], which proposes replacing attention with Grassmann-flow geometric mixing over token subspaces. Tokens are projected into a lower-dimensional subspace, causal interactions between positions are encoded through Plücker coordinates, and a learned geometric mixer replaces attention in each block. Rather than only reproducing the mechanism, I evaluate it at a nontrivial scale (~ 100M parameters).

This report compares the Grassmann model against a causal Transformer baseline trained with the same tokenizer and data pipeline. I report held-out loss and perplexity, total training time, inference runtime, and profiled compute scaling.

While the core project remains the same as in the proposal, the final evaluation setup differs from the original plan: instead of external benchmarks such as WikiText-103 and Lambada, I evaluate both models on a shared held-out split drawn from the same FineWeb-derived token stream used in training.

2 Related Work

The main reference for the Grassmann model is the original paper by Chong et al. [1], which introduces the idea of replacing self-attention with causal geometric mixing over subspace representations. For the baseline, I use a GPT-style causal Transformer, since it remains the standard architecture for language modeling at this parameter scale. Training data comes from a FineWeb-derived corpus [2], a commonly used large-scale web-text dataset.

Earlier work on language-modeling benchmarks has also shown that prediction quality can depend on long-range discourse context rather than only local n-grams [3]. I kept this in mind when designing the evaluation: both models share the same tokenizer and data pipeline and are trained on the exact same sequence of tokens, making the comparison as direct as possible.

3 Method

3.1 Model setup

Both models use the GPT-2 tokenizer ($50{,}257$ vocabulary items), context length 1024, and tied input/output token embeddings. The main Grassmann run has approximately 102.47M parameters, with:

- $d_{\text{model}} = 640$, $n_{\text{layers}} = 16$, $d_{\text{ff}} = 2560$,
- geometric rank $r = 24$, projection width $d_{\text{geom}} = 256$,
- causal offsets $\{1, 2, 4, 8, 16, 32, 64\}$.

The Transformer baseline is size-matched and trained with the same data pipeline and optimizer schedule for a fair comparison.

3.2 Causal Grassmann block

For each token position t, the model computes geometric features only from past offsets $t - \Delta$, preserving autoregressive causality. Given reduced vectors $z_{t-\Delta}$ and z_t, pairwise antisymmetric products define a Plücker feature vector:

$$p_{ij}^{(\Delta)}(t) = z_{t-\Delta,i} z_{t,j} - z_{t-\Delta,j} z_{t,i}, \quad i < j.$$

These normalized geometric features are projected back to model width, aggregated across offsets, and mixed with the current hidden state through a learned gate, followed by a standard FFN residual path.

3.3 Why this behaves like attention

The description of what the "Grassmann" LM architecture above explains what the block computes, but it is not clear why this can replace attention. In a standard causal attention layer, the current token forms a query, compares it against previous keys, and then uses those similarity scores to build a weighted combination of previous values. The output is therefore content dependent: the representation at position t changes depending on which earlier tokens are most compatible with the current token.

The Grassmann block keeps that same high level idea of content-dependent retrieval, but changes the mechanism. Instead of scalar query-key scores, it compares the current reduced vector z_t with a small set of previous reduced vectors $z_{t-\Delta}$ through antisymmetric Plücker coordinates. These coordinates describe the oriented two-dimensional subspace spanned by the pair $(z_{t-\Delta}, z_t)$. Intuitively, the model is not asking "how similar are these two tokens?" but rather "what geometric relation do these two token directions form together?" A learned projection then maps that relation back into the model space.

This creates attention-like behavior in three steps. First, the geometric feature for offset Δ depends on both the current token and the earlier token, so the interaction is still content dependent rather than fixed. Second, different offsets can produce different projected updates, so the model can emphasize

different kinds of past context depending on the present token. Third, the gate combines the geometric update with the current hidden state, which plays a role similar to deciding how much retrieved contextual information should influence the next representation.

There is also an important difference from Transformer attention. My implementation does not score *all* previous positions and normalize them with a softmax. It only looks at the fixed causal offsets $\{1, 2, 4, 8, 16, 32, 64\}$, computes one Plücker-based interaction for each available offset, and averages the resulting updates. So the Grassmann mixer is attention-like because it performs token-dependent contextual mixing, but it is not full attention: it is a sparse geometric retrieval rule over selected past positions. This is exactly why it can be cheaper and more structured, but it is also why long-range information outside those offsets is missed.

3.4 Training and evaluation protocol

Both models are trained with the same tokenization, sequence length, and held-out evaluation windows so that the comparison is controlled. I measure held-out loss and perplexity on the same test windows for both models. I also run context-sensitivity sweeps, total runtime benchmarks, and profiled FLOP counts at different prompt lengths. For compute profiling, both models use KV-cached decode so the numbers reflect actual autoregressive inference.

4 Experiments

4.1 Datasets and data format

- **Training data:** FineWeb-derived token stream consisting of a 100B-token sample, downloaded using the recommended method on the dataset's Hugging Face page and then used in GPT-2-tokenized, pre-tokenized form [2].
- **Held out evaluation:** Fixed token windows from `data/test_tokens.bin` with starts from `data/test_starts.npy`.
- **Evaluation approach:** shared sequence length 1024, device CUDA, bfloat16 evaluation.

4.2 Benchmarks

- `eval.py` measures held-out loss/perplexity, a short context sweep, and prompt-generation speed on the same checkpoints.
- `bench.py` measures synthetic-prompt prefill latency and uncached decode throughput across prompt lengths.
- `compute_profile.py` profiles prefill and KV-cached decode FLOPs as prompt length increases.

4.3 Head to head summary

Table 1 summarizes the main checkpoint metrics from `polished_metrics.csv`. The Grassmann model performs better on held-out loss and perplexity, evaluates faster on the shared held-out set, and reaches its best context-bucket loss at the full 1024-token window.

Table 1: Main comparison metrics at step $9{,}500$, taken directly from `polished_metrics.csv`. Grassmann lowers held-out perplexity by about 24.6%, while the Transformer remains much faster in greedy generation throughput.

Model	Params (M)	Loss	Perplexity	Eval sec	Best ctx	Greedy tok/s
Grassmann	102.47	5.8413	344.23	528.09	1024	73.57
Transformer	111.60	6.1234	456.41	835.58	256	197.31

4.4 Training time and runtime comparison

Training was noticeably faster for the Grassmann model: $12{:}42$ (hours:minutes) versus $24{:}06$ for the Transformer. That is about $1.90\times$ faster, or a 47.3% reduction in total training time.

Inference shows a different pattern. At short prompts the Transformer is clearly faster, but the gap shrinks as context length grows and reverses at 1024 tokens in my benchmark.

Figure 1 comes from `bench.py`, which uses synthetic token prompts, batch size 1, and 64 generated tokens per decode run. The plot shows a mixed result: the Transformer is much faster for short prompts and still leads on uncached decode throughput, but Grassmann's prefill curve grows more slowly and is faster at the longest tested prompt (24.96 ms vs. 36.22 ms at length 1024).

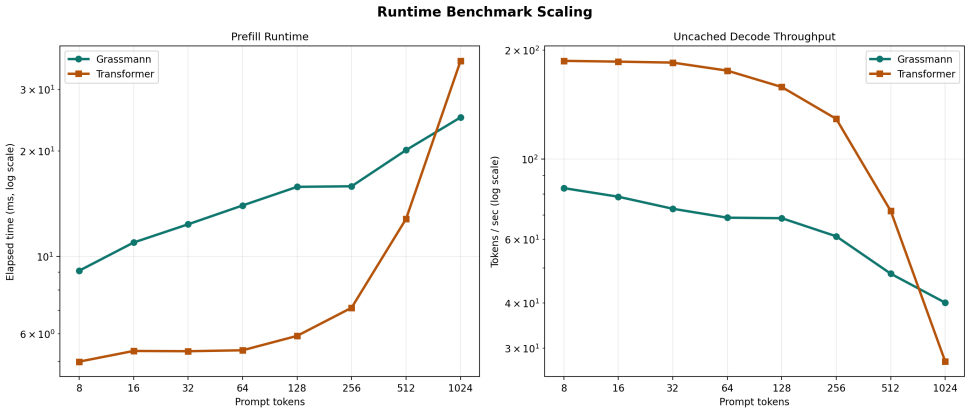

Figure 1: Runtime scaling from `bench.py`. Left: prefill latency versus prompt length. Right: uncached decode throughput for 64 generated tokens.

There is therefore a clear trade-off. The Transformer is faster at short contexts, likely because attention kernels are currently better optimized. Grassmann catches up as the prompt grows longer and is faster on prefill at 1024 tokens. However, the practical speed advantage only appears near the top of the tested range. Further experiments with substantially larger context windows would be needed to determine whether the Grassmann model realizes a more decisive linear-scaling advantage over the Transformer's quadratic prefill cost.

4.5 Context sensitivity comparison

Both models were evaluated by scoring a fixed suffix while varying the visible prefix length. Grassmann is more stable across context lengths, while the Transformer shows a larger improvement as context increases before flattening out.

Grassmann has better loss at every tested context length, from 5.8317 at 32 tokens to 5.8302 at 1024. At the same time, its numbers barely change as context grows: the spread from 32 to 1024 is very small. The Transformer, by contrast, improves more noticeably from 6.1180 at 32 to 6.0990 at 256 before leveling off and slightly worsening again by 1024. This suggests that the Grassmann model may not be using long-range context as effectively as the Transformer, even though its absolute quality is better.

This matters because prior work [3] has shown that some predictions genuinely depend on long-range discourse context rather than only local patterns. That concern is also consistent with the current implementation: the model uses predefined look-behind offsets, so it cannot directly mix information from arbitrarily far back in the sequence. This is a real limitation at larger context windows, because tokens outside that offset pattern are effectively invisible to the geometric mixer.

4.6 Profiled compute scaling with KV cache

To separate raw runtime from operation-level compute cost, I also profiled FLOPs as prompt length increases. Figure 2 comes from `compute_profile.py`; its prefill panel runs out to 1024 tokens, while the cached decode panel profiles 16 generated steps up to prompt length 256. The decode study uses the cached autoregressive path for both models, so it reflects inference with KV cache rather than naive recomputation. This matters because, without KV caching, my implementation of the Grassmann-flow model would drift back toward quadratic behavior rather than the intended

linear-in-context decode path. The two models track each other closely in both panels: at length 1024, prefill is 254.64 GFLOPs for Grassmann versus 270.07 GFLOPs for the Transformer, while at decode length 256 Grassmann is slightly higher (66.40 vs. 62.92 GFLOPs).

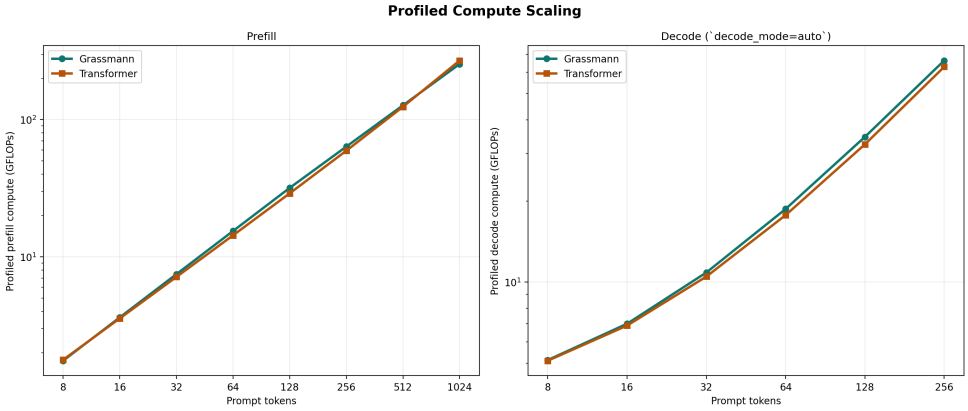

Figure 2: Profiled compute scaling from `compute_profile.py`. Left: prefill FLOPs versus prompt length. Right: KV-cached decode FLOPs for 16 generated steps.

These profiles are worth examining carefully because they tell a somewhat different story from the overall runtime numbers. When both models use KV cache, the FLOP difference between them is quite small. Prefill FLOPs are close, and cached decode FLOPs are also close, with Grassmann even slightly higher at some prompt lengths. In these experiments, the main benefit of Grassmann is therefore not lower inference FLOPs, but better quality and faster training.

4.7 Discussion

The final comparison shows that Grassmann flow is a viable attention-free alternative at this scale. Its main benefits in this project are:

- better quality on held-out data than the size-matched Transformer baseline; held-out perplexity improves by about 24.6%
- substantially shorter total training time in the current implementation,
- competitive long context runtime, with an advantage appearing at the longest tested prompt.

That said, there are clear caveats. Once both models use KV cache, the FLOP difference during decode is small, so the Grassmann model does not have a large practical inference-compute advantage. It is also slower than the Transformer at short contexts, and the context-sensitivity results suggest that it may not be taking full advantage of long-range information in this checkpoint.

Overall, geometric mixing can outperform a standard Transformer on quality and training efficiency at 100M parameters, but it is not an obvious win on inference cost. It is difficult to tell how much of that is fundamental to the architecture versus specific to my implementation. Testing with a larger context window would be a natural next step, since 1024 tokens is still relatively short and the linear-versus-quadratic scaling difference should become more pronounced on longer sequences.

One possible niche is smaller models that do not require very large effective contexts. In that setting, Grassmann flow may offer better training speed while still improving the quality of early-token predictions. However, the practical use cases remain unclear in the current state of the model. In my benchmarks, the advantages over a classical Transformer appear only in some long-context settings, and the fixed-offset geometric mixer cannot directly incorporate information from the earliest tokens once they fall outside the offset pattern.

5 Supplementary Material

Additional materials for the report:

- Training, evaluation, and benchmarking code written entirely by me (the original paper did not provide implementation code),
- Benchmark logs,
- Demo video
- GitHub repository

References

[1] Z. Chong, "Attention is not what you need," 2025. [Online]. Available: https://arxiv.org/abs/2512.19428

[2] G. Penedo, H. Kydlíček, L. B. allal, A. Lozhkov, M. Mitchell, C. Raffel, L. V. Werra, and T. Wolf, "The fineweb datasets: Decanting the web for the finest text data at scale," in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. [Online]. Available: https://openreview.net/forum?id=n6SCkn2QaG

[3] D. Paperno, G. Kruszewski, A. Lazaridou, Q. N. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, and R. Fernández, "The lambada dataset: Word prediction requiring a broad discourse context," 2016. [Online]. Available: https://arxiv.org/abs/1606.06031